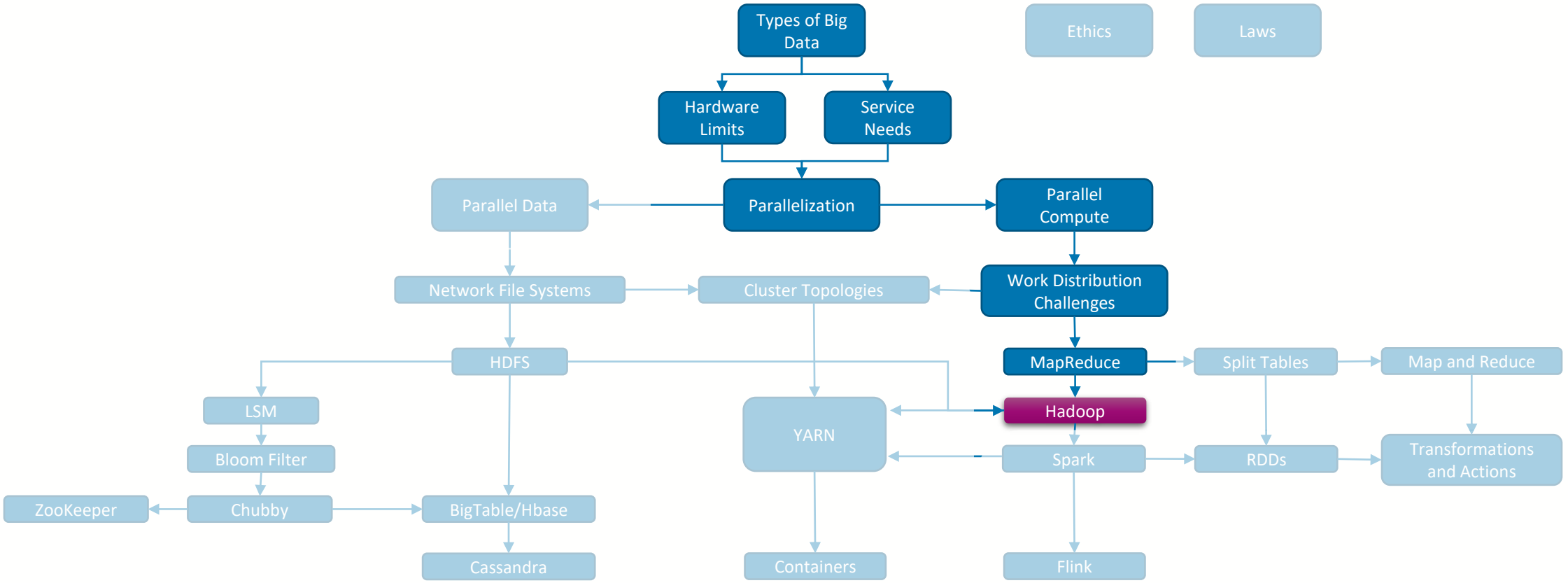
W2-02: Hadoop

COMPSCI4064 (H) and COMPSCI5088 (M)

Dr. Richard McCreadie

University of Glasgow

Where are we?



University of Glasgow

MapReduce as Closed Source

- Importantly, Google did not release an implementation of MapReduce as open source, they simply published the design and reasoning

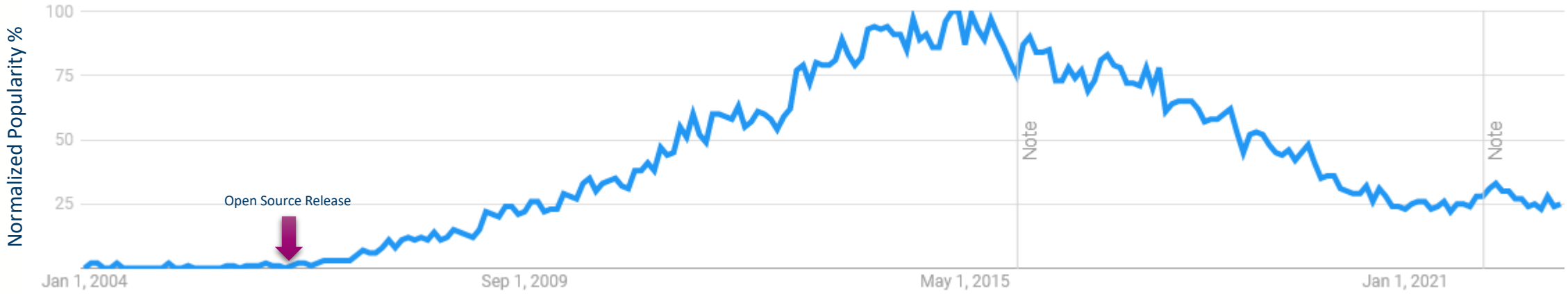
- For this reason, MapReduce did not immediately take-off
  - Instead companies seeing the utility of the idea started implementing their own versions internally
  - … including one of Google's main competitors at the time Yahoo!

- Yahoo! released their implementation of MapReduce as open source in April 2006
  - … and called it

University of Glasgow

What is hadoop

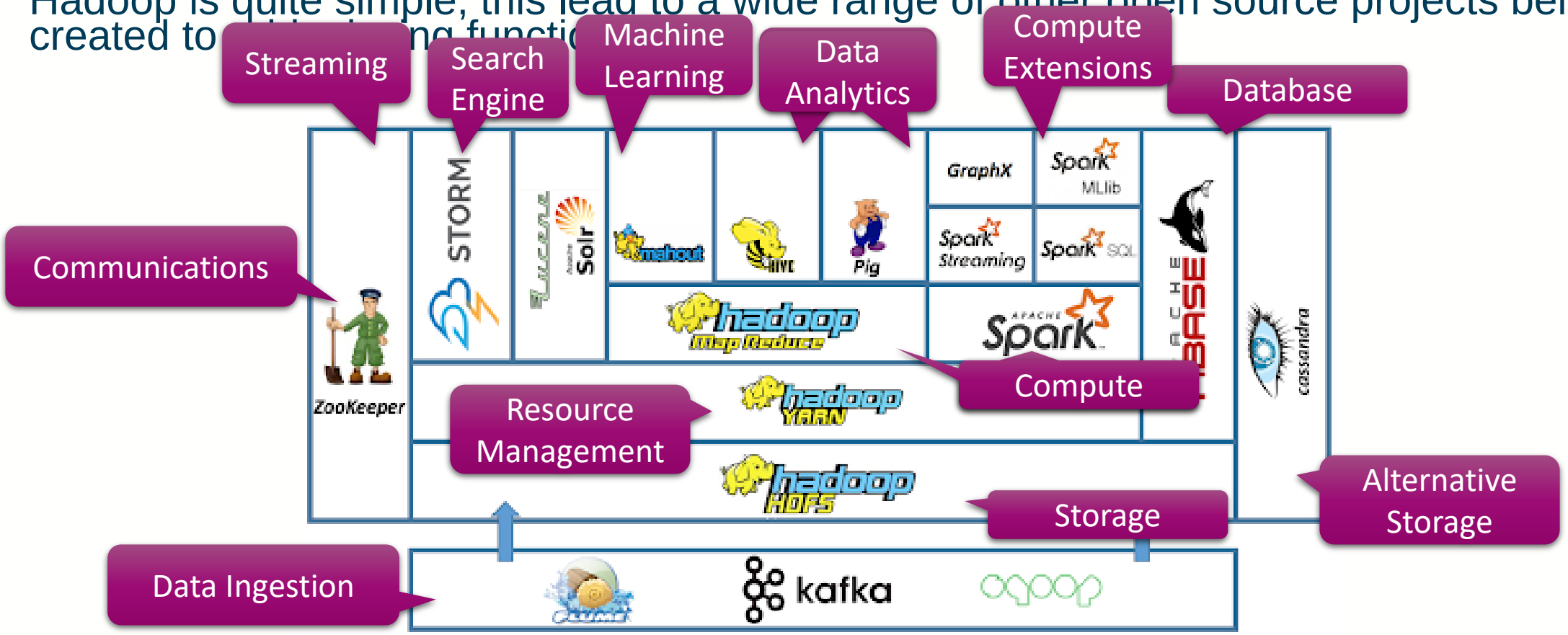
- Hadoop (v1) was an implementation of the MapReduce paper in Java, along with a distributed file store called the HDFS
  - It's a lot of other things these days, but here we will be focusing on the MapReduce component on Hadoop in these slides
  - Still in use today, although its losing popularity as developers continue to transition to Spark (covered later)



Google Trends Chart for Hadoop

An aside on the Hadoop Ecosystem

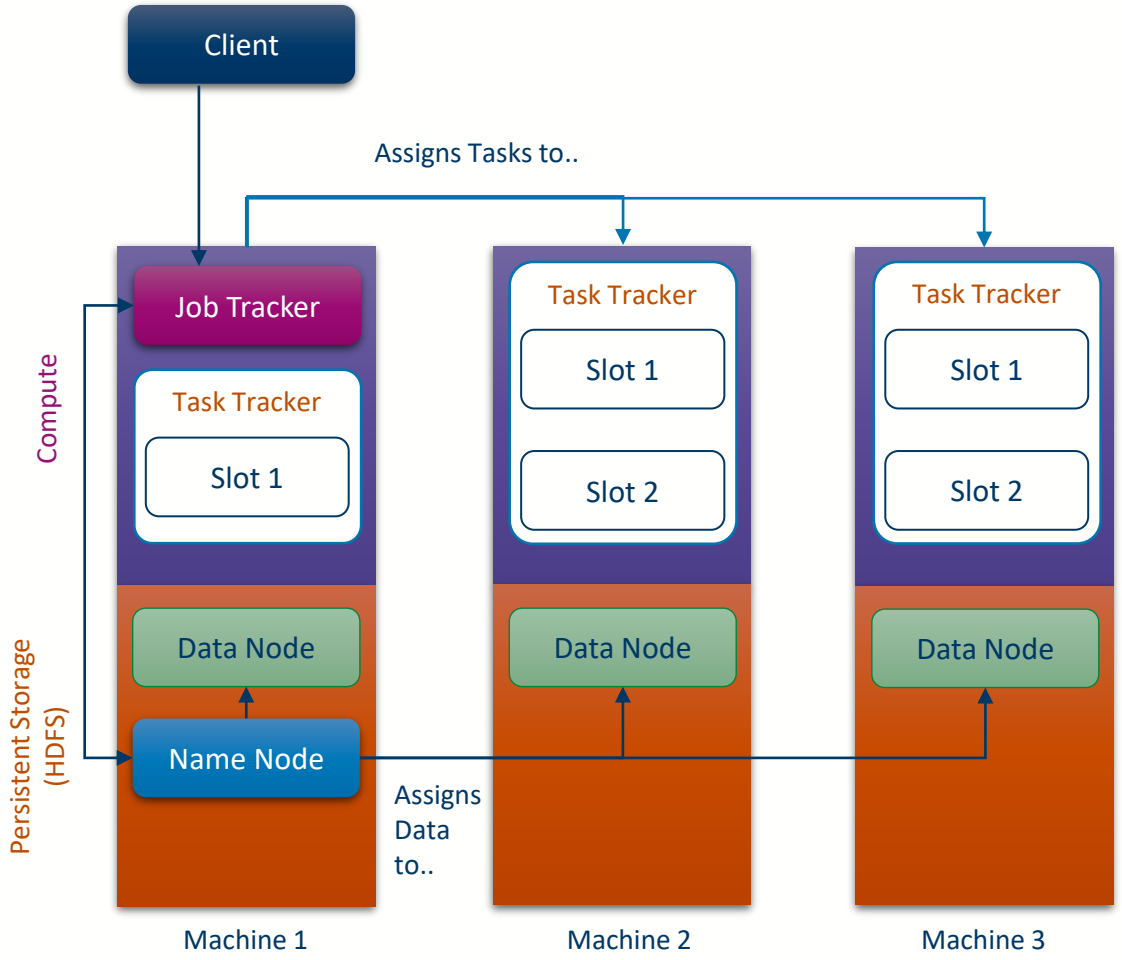
- Hadoop had a large impact on how Big Data processes were developed, but the core of Hadoop is quite simple, this lead to a wide range of other open source projects being created to



University of Glasgow

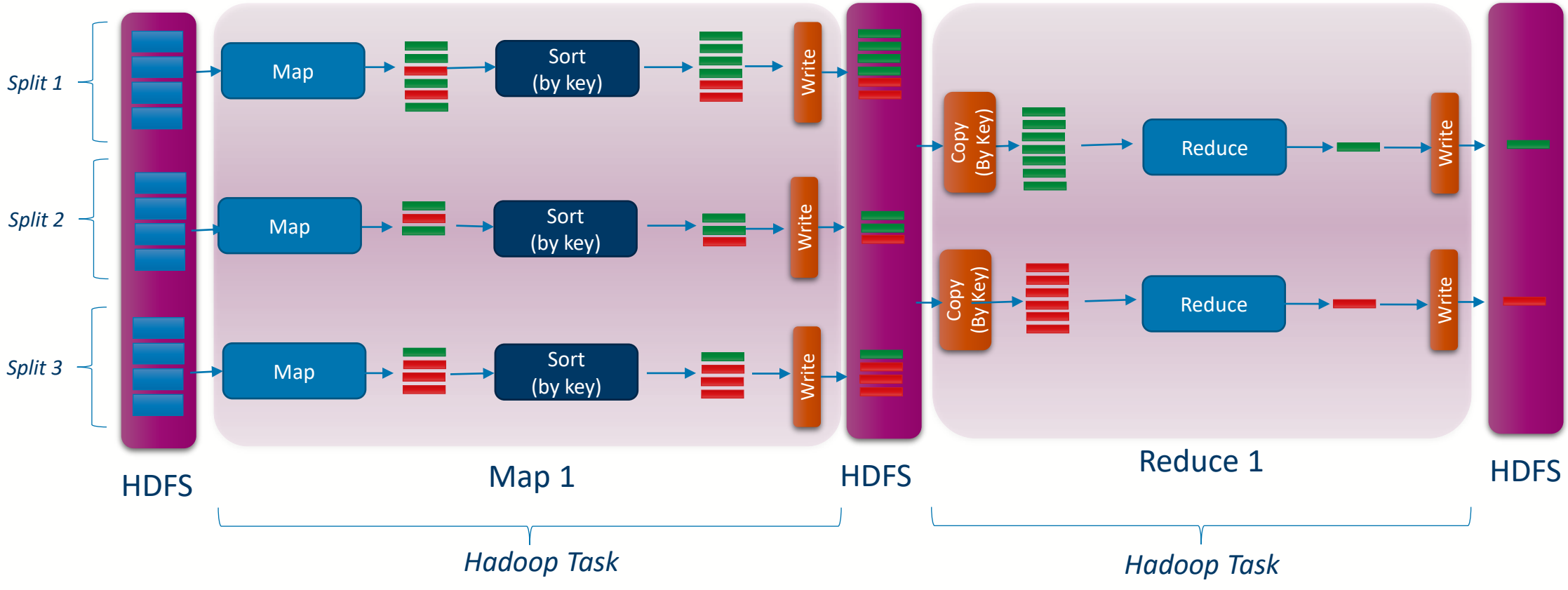
Hadoop Architecture

- Hadoop Compute
  - Client: Software that the user uses to submit work to the Hadoop cluster, communicates with the Job Tracker
  - Job Tracker: Receives work from the client, uses the Name Node to identify where data is located, assigns work to task trackers
  - Task Tracker: Executes Map and Reduce tasks
- Hadoop HDFS
  - Name Node: Tracks the location of all data held on the HDFS, handles replication
  - Data Node: Handles storage and access of the local files on the machine



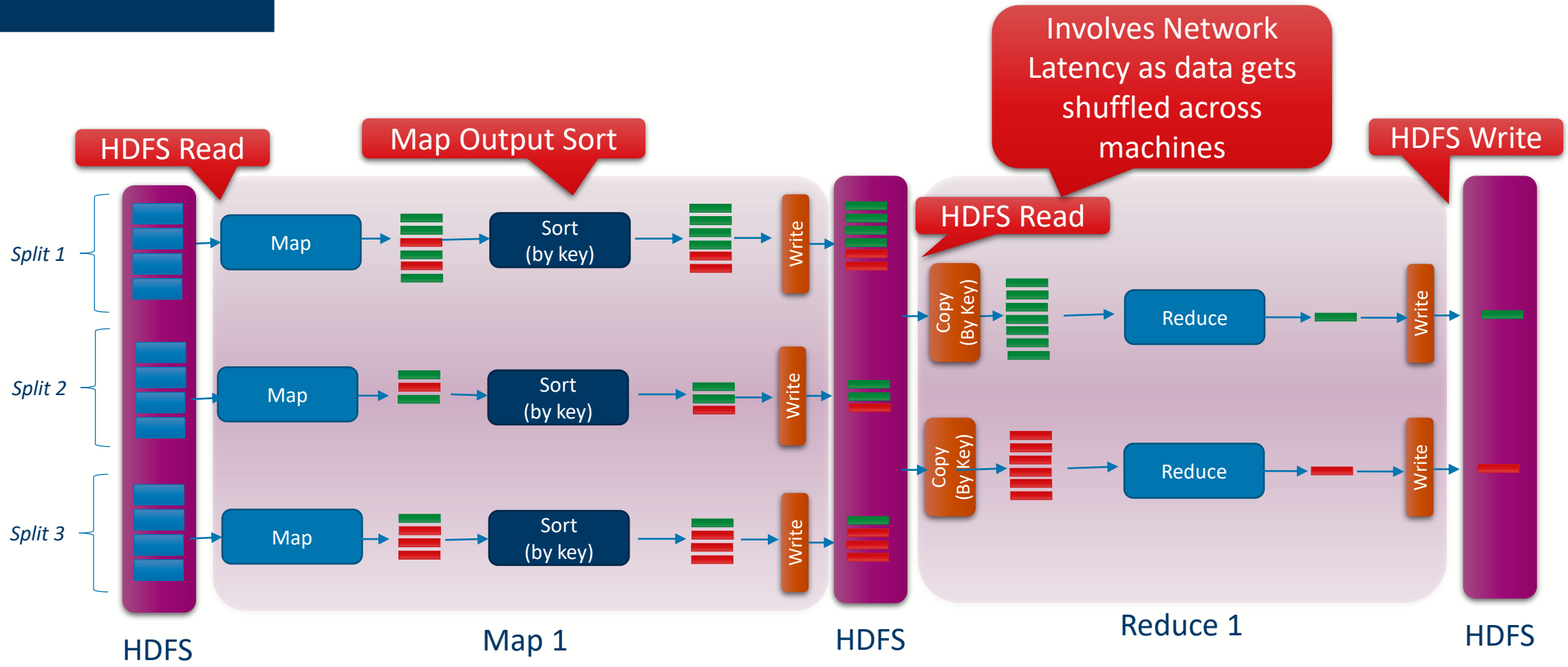
MapReduce in Hadoop

- Hadoop's implementation of MapReduce strictly follows the original MapReduce design, using the HDFS for intermediate storage



University of Glasgow

Overheads in Hadoop MapReduce



University of Glasgow

Data Locality

- One of the key tenants of Hadoop is to move the compute to the data, as the alternative is more expensive

- To enable this, Hadoop will choose task trackers to submit work to based on whether they have the data needing processed locally
  - Map tasks look to see where the input splits are stored
  - Reduce tasks check to see where the items for each key are stored
  - The Job Tracker communicates with the Name Node to find out where the data the task needs is currently located

University of Glasgow

Work Distribution Challenges

- Load Balancing: Not handled natively, but can be controlled via data splitting
- Overheads
  - New Stages: Hadoop performs poorly here, it always needs a reduce phase, and the intermediate sort and write to the HDFS is costly
  - Transfer Latency: The HDFS is optimised for block reads of data, so latencies are manageable for large reads, but terrible for small reads. Hadoop does try and avoid network costs by moving the compute to the data where possible
- Orchestration
  - Task Assignment: Handled automatically by the Job Tracker
  - Machine Failures: The Job Tracker will re-start work on free Task Trackers
  - Master Failures: Usually no recovery possible, unless you have multiple Job trackers
  - Network Failures: Treated as Machine Failures.

University of Glasgow

Hadoop Versioning

- V1
  - Basic Implementation of the Hadoop framework
- V2
  - Introduces YARN (Yet Another Resource Negotiator)
    - Provides better resource allocation and scheduling
- V3
  - Adds multiple Name Node Support
  - Reduced read/write costs via erasure coding
  - Adds Container and GPU support

University of Glasgow

Course Coordinator
Dr. Richard McCreadie

Email: richard.mccreadie@glasgow.ac.uk
Room: SAWB 304

#UofGWorldChangers
@UofGlasgow